

Membrane Neuron Simulator

Formulation and implementation of a static solver for fluid-driven membrane neurons

W. J. van Vuuren

Technical reference accompanying the source code

<https://github.com/vanvuurenwerner2305-blip/MechanicalNeuronSimulation>

Abstract

This document describes the mathematics and algorithms of the Membrane Neuron Simulator, the software used to simulate the mechanical neurons of the accompanying thesis. A mechanical neuron consists of thin rubber membranes that separate fluid chambers and deform against rigid parts. The software imports such a device from CAD, models the membranes as geometrically nonlinear shells, the chambers as volume-dependent pressure laws and the rigid parts as contact obstacles, and solves for static equilibrium by minimising a total potential energy with a safeguarded Newton method. Sweeps over the input pressures map the neuron's response, and a whole neuron can be characterised over a grid of its parameters and kept as a dataset.

None of the ingredients is new: the membrane and bending elements, the pressure loads, the penalty contact and the Newton globalisation are standard techniques from the literature, cited where they are used. The purpose of this document is to state exactly which formulation the code implements, so that the results in the thesis can be reproduced and checked.

Contents

1	Scope and overview	3
2	Equilibrium as energy minimisation	3
3	Membrane energy	4
4	Bending energy	5
5	Fluid chambers	6
5.1	Volume change from the membranes alone	6
5.2	Pressure laws	6
5.3	Potential, load and tangent	7
5.4	Load path	7
6	Contact with rigid bodies	7
6.1	Exact signed distance	7
6.2	Penalty energy	8
7	Contact between membranes	8
8	Assembly and exact derivatives	9
9	Newton iteration	10
9.1	Line search on the energy	10
9.2	Second-order volume correction	11

10 Load stepping	11
11 From CAD to model	11
12 Parameter sweeps	12
13 The activation-function device	12
13.1 Roles	13
13.2 Solid tube	13
13.3 Free rigid bodies	13
13.4 Deformable parts	14
13.5 Chamber walls on a solid	14
13.6 The tube closing on itself	15
13.7 Cross-sectional area	15
13.8 Flow network	15
13.9 Structure–flow iteration	15
13.10The outputs	16
13.11A design as a part of a neuron	16
14 The full neuron and its characterisation	17
15 Verification	18
16 Limitations	19

1 Scope and overview

The simulator answers one question: *given a device and the pressures in its input chambers, what is the static equilibrium shape of its membranes, and what are the resulting pressures in its sealed chambers?* The device is built in CAD (Fusion 360 in practice), exported as a STEP file, and every solid body is given one of the following roles:

- **Membrane:** a thin solid with membrane stiffness only;
- **Shell:** a thin solid with membrane and bending stiffness;
- **Rigid body:** an obstacle that the membranes cannot penetrate;
- **Fluid chamber:** a solid that fills a fluid space, with a pressure model (constant-pressure input, sealed ideal gas, sealed liquid, or vent to the surroundings);
- **Ignore.**

Chamber walls are never meshed. Rigid bodies are meshed as closed surfaces, and membranes and shells are replaced by triangulated mid-surfaces.

The whole analysis is *quasi-static*. Nothing depends on time: there is no inertia, no viscosity and no fluid flow. Every chamber is a lumped volume with one uniform pressure. This matches how the neuron is analysed in the thesis, where the input pressures are held at steady state and the neuron’s output is the equilibrium pressure of its sealed pre-activation chamber.

The application has three spaces. *Inputs to pre-activation* is the neuron itself (everything up to Section 12). *Pre-activation to activation* is the activation-function device, a membrane that squeezes a gas-carrying tube (Section 13); it is simulated once over its pre-activation range and saved as a *design*. *Full neuron* links a neuron to a design and characterises the whole neuron over a grid of its parameters, so that it can later be used without being simulated again (Section 14).

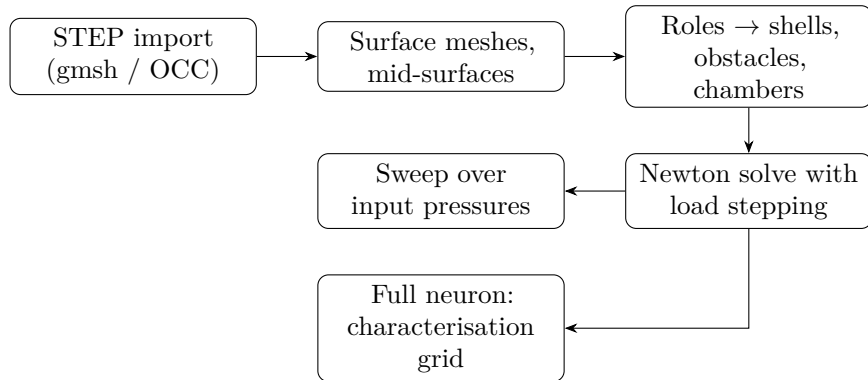


Figure 1: Pipeline of the simulator (Sections 11, 2–10, 12 and 14).

Units and conventions. The application works in millimetres, newtons and megapascals (so energies are in N mm), and it reports pressures in kPa. All pressures are *gauge* pressures: the surroundings are at 0. A membrane face that touches no chamber therefore sees zero pressure. The solver core (`membrane_sim`) itself has no units; it works in whatever consistent system the caller uses.

Code map. Table 1 links the sections of this document to the source files.

2 Equilibrium as energy minimisation

Let $\mathbf{u} \in \mathbb{R}^n$ collect all unknowns: the current position $\mathbf{x}_a \in \mathbb{R}^3$ of every mesh node a of every shell and, for shells with bending, one rotation ψ_e per mesh edge (Section 4). The rest positions

File	Contents	Section
membrane_sim/shell.py	membrane and bending elements	3, 4
membrane_sim/fluid.py	chamber volumes and pressure laws	5
membrane_sim/contact.py	rigid obstacles, signed distance	6
membrane_sim/shell_contact.py	contact between membranes	7
membrane_sim/solver.py	assembly and Newton iteration	8, 9
membrane_sim/environment.py	load stepping	10
membrane_sim/empirical.py	a membrane replaced by a design	13.11
app/cad.py, app/builder.py	CAD import, meshing, coupling	11
app/sweep.py	parameter sweeps	12
membrane_sim/solid.py, flow.py, ...	device parts, gas flow	13
app/activation.py	device study, outputs, stored solution	13
app/full_neuron.py	linking, characterisation dataset	14
app/analysis.py	slices of the dataset for plotting	14

Table 1: Where each part of the formulation lives in the code.

are $\mathbf{X}_a$. Nodes on a clamped boundary are fixed at their rest positions and removed from the unknowns.

Every force in the model is conservative, so the static equilibrium is a stationary point of the total potential energy

$$\Pi(\mathbf{u}; \lambda) = \sum_{\text{shells}} \left(U_{\text{mem}}(\mathbf{u}) + U_{\text{bend}}(\mathbf{u}) \right) + E_{\text{rigid}}(\mathbf{u}) + E_{\text{sheet}}(\mathbf{u}) - \sum_{\text{chambers } v} \Phi_v(\Delta V_v(\mathbf{u}); \lambda), \quad (1)$$

where U_{mem} and U_{bend} are the strain energies of the shells, E_{rigid} and E_{sheet} are penalty energies for contact with rigid bodies and between membranes, ΔV_v is the volume change of chamber v , Φ_v is the work done by its fluid, and $\lambda \in [0, 1]$ is the load factor used to ramp the pressures. The solver finds a point where

$$\mathbf{R}(\mathbf{u}) = \frac{\partial \Pi}{\partial \mathbf{u}} = \mathbf{0}, \quad (2)$$

using the tangent $\mathbf{K} = \partial^2 \Pi / \partial \mathbf{u}^2$, and it uses Π itself as the merit function of its line search (Section 9). Stable equilibria are local minima of Π . Because the formulation is variational, $\mathbf{K}$ is symmetric.

3 Membrane energy

Each shell is a mesh of three-node triangles with constant strain (CST). For a triangle with rest vertices $\mathbf{X}_0, \mathbf{X}_1, \mathbf{X}_2$, an orthonormal in-plane basis is built from its rest normal $\mathbf{N}_0$:

$$\mathbf{e}_1 = \frac{\mathbf{X}_1 - \mathbf{X}_0}{\|\mathbf{X}_1 - \mathbf{X}_0\|}, \quad \mathbf{e}_2 = \mathbf{N}_0 \times \mathbf{e}_1, \quad \mathbf{D}_m = \begin{bmatrix} \mathbf{e}_1 \cdot (\mathbf{X}_1 - \mathbf{X}_0) & \mathbf{e}_1 \cdot (\mathbf{X}_2 - \mathbf{X}_0) \\ \mathbf{e}_2 \cdot (\mathbf{X}_1 - \mathbf{X}_0) & \mathbf{e}_2 \cdot (\mathbf{X}_2 - \mathbf{X}_0) \end{bmatrix}. \quad (3)$$

With the current edge matrix $\mathbf{D}_s = [\mathbf{x}_1 - \mathbf{x}_0, \mathbf{x}_2 - \mathbf{x}_0] \in \mathbb{R}^{3 \times 2}$, the surface deformation gradient and the right Cauchy–Green tensor are

$$\mathbf{F} = \mathbf{D}_s \mathbf{D}_m^{-1} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{C} = \mathbf{F}^\top \mathbf{F} \in \mathbb{R}^{2 \times 2}. \quad (4)$$

This is exact for arbitrarily large displacements and rotations [2]. With A_0 the rest area and t the thickness, two materials are available.

Incompressible neo-Hookean (default, rubber). For an incompressible sheet in plane stress, the through-thickness stretch is $\lambda_3 = 1/J$ with $J^2 = \det \mathbf{C}$ (the area stretch). Substituting it into the three-dimensional invariant $I_1 = \text{tr } \mathbf{C} + \lambda_3^2$ gives the energy per element

$$U_{\text{mem}}^e = A_0 \frac{\mu t}{2} \left(\text{tr } \mathbf{C} + \frac{1}{\det \mathbf{C}} - 3 \right) + T A, \quad \mu = \frac{E}{3}, \quad (5)$$

where $\mu = E/3$ is the shear modulus of an incompressible material ($\nu = 1/2$). The thickness change is eliminated analytically, so no extra unknown is needed.

St. Venant–Kirchhoff. On the Green–Lagrange strain $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I})$, with the plane-stress Lamé constant,

$$U_{\text{mem}}^e = A_0 t \left(\frac{\bar{\lambda}}{2} (\text{tr } \mathbf{E})^2 + \mu \mathbf{E} : \mathbf{E} \right) + T A, \quad \mu = \frac{E}{2(1+\nu)}, \quad \bar{\lambda} = \frac{E\nu}{1-\nu^2}. \quad (6)$$

This model is only meaningful for small strains. It is kept for comparison with linear plate theory.

Pre-tension. In both cases $T A$ adds an optional isotropic in-plane pre-tension T (force per length) times the *current* element area $A = \frac{1}{2} \|(\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0)\|$. It acts like a surface tension. It is zero by default.

Remark. A membrane without pre-tension or bending stiffness has no transverse stiffness in its flat state: the tangent is singular at the start of every solve. The load stepping and the damped Newton steps of Section 9 deal with this. It is also why a slack membrane’s response is so compliant near zero pressure difference: a clamped circular membrane under pressure p (Hencky’s problem [8]) deflects as $w \propto p^{1/3}$, so its displaced volume per unit pressure behaves like $p^{-2/3}$.

4 Bending energy

The **Shell** role adds bending stiffness through a geometrically nonlinear version of Morley’s constant-moment triangle [12], written with mid-edge normals as in discrete shell models [5, 6]. Every mesh edge e carries one extra unknown, the rotation ψ_e of its mid-edge normal. The first attempt used an “average face normal” hinge without ψ_e . That model’s bending response depended on the orientation of the mesh, and the ψ_e unknowns remove that dependence. A test checks this (`test_bending_energy_is_independent_of_mesh_orientation`).

Edge rotations. For triangle f with current normal $\mathbf{n}_1$ and edge k (opposite vertex k , running from vertex i to vertex j with unit direction $\hat{\mathbf{e}}$), the signed angle between $\mathbf{n}_1$ and a second normal $\mathbf{n}_2$ about the edge is

$$\theta_k = \text{atan2}((\mathbf{n}_1 \times \mathbf{n}_2) \cdot \hat{\mathbf{e}}, \mathbf{n}_1 \cdot \mathbf{n}_2). \quad (7)$$

The rotation ϕ_k of the mid-edge normal relative to face f depends on the edge type:

$$\phi_k = \begin{cases} \frac{1}{2}\theta_k + s_k \psi_e, & \text{interior edge, } \mathbf{n}_2 = \text{normal of the neighbouring face,} \\ \theta_k, & \text{clamped boundary edge, } \mathbf{n}_2 = \mathbf{N}_0 \text{ (rest normal),} \\ \psi_e, & \text{free (hinged) boundary edge.} \end{cases} \quad (8)$$

Here $s_k = \pm 1$ records the direction in which face f traverses the edge, so the two faces that share an interior edge see the same mid-edge normal. Here θ_k is the full dihedral angle, so $\frac{1}{2}\theta_k$ puts the mid-edge normal halfway between the two faces, and ψ_e lets it rotate further. A clamped edge is one whose two end nodes are both fixed while the shell’s boundary rotation is set to “clamped”. Its mid-edge normal is held at the rest normal, and its ψ_e is removed from the unknowns.

Curvature and energy. The face has the constant curvature tensor

$$\boldsymbol{\kappa} = \frac{1}{A_0} \sum_{k=1}^3 l_k \phi_k \mathbf{m}_k \otimes \mathbf{m}_k, \quad (9)$$

with rest edge lengths l_k and in-plane outward edge normals $\mathbf{m}_k$ taken from the rest state. With the change of curvature $\Delta\boldsymbol{\kappa}$ measured from the rest state ($\Delta\phi_k = \phi_k - \phi_k^0$, so curved CAD geometry is stress-free), the energy of a linear-elastic plate,

$$U_{\text{bend}}^e = A_0 \frac{D}{2} \left(\nu (\text{tr } \Delta\boldsymbol{\kappa})^2 + (1 - \nu) \Delta\boldsymbol{\kappa} : \Delta\boldsymbol{\kappa} \right), \quad D = \frac{Et^3}{12(1 - \nu^2)}, \quad (10)$$

becomes a quadratic form in $\Delta\boldsymbol{\phi}$. With $\text{tr}(\mathbf{m}_i \otimes \mathbf{m}_i) = 1$ and $(\mathbf{m}_i \otimes \mathbf{m}_i) : (\mathbf{m}_j \otimes \mathbf{m}_j) = (\mathbf{m}_i \cdot \mathbf{m}_j)^2$,

$$U_{\text{bend}}^e = \Delta\boldsymbol{\phi}^\top \mathbf{M} \Delta\boldsymbol{\phi}, \quad M_{ij} = \frac{D}{2A_0} l_i l_j \left(\nu + (1 - \nu)(\mathbf{m}_i \cdot \mathbf{m}_j)^2 \right). \quad (11)$$

$\mathbf{M}$ depends only on the rest geometry and is precomputed. All the geometric nonlinearity sits in $\phi_k(\mathbf{u})$. For the neo-Hookean material $\nu = \frac{1}{2}$, so $D = Et^3/9$.

5 Fluid chambers

5.1 Volume change from the membranes alone

A chamber is bounded partly by rigid walls and partly by membranes. Only its *change* in volume matters, and it can be computed from the membranes alone. Every membrane is clamped along its boundary, so the rest mesh and the deformed mesh of the same membrane together close a (possibly self-intersecting) volume, and that volume is exactly the volume the membrane has swept. Using signed cone volumes about any fixed point $\mathbf{o}$,

$$V_{\text{cone}}(\mathbf{x}) = \frac{1}{6} \sum_{\text{faces}} (\mathbf{x}_0 - \mathbf{o}) \cdot ((\mathbf{x}_1 - \mathbf{o}) \times (\mathbf{x}_2 - \mathbf{o})), \quad (12)$$

the volume change of chamber v is

$$\Delta V_v(\mathbf{u}) = \sum_{\text{shells } s \text{ on } v} \sigma_{s,v} (V_{\text{cone}}(\mathbf{x}_s) - V_{\text{cone}}(\mathbf{X}_s)), \quad (13)$$

where $\sigma_{s,v} = +1$ if the shell's normal points out of the chamber (the chamber lies behind the shell and pushes it along its normal) and -1 otherwise. The result does not depend on $\mathbf{o}$ or on the rigid walls, which is why chamber walls never need a mesh. The code uses the centroid of the rest mesh for $\mathbf{o}$, which limits round-off.

ΔV_v is a cubic polynomial in the node coordinates. Its gradient is $\mathbf{g}_v = \partial \Delta V_v / \partial \mathbf{u}$, which per face is $\frac{1}{6}(\mathbf{x}_1 \times \mathbf{x}_2, \mathbf{x}_2 \times \mathbf{x}_0, \mathbf{x}_0 \times \mathbf{x}_1)$ (relative to $\mathbf{o}$). Its Hessian is constant per face.

5.2 Pressure laws

Each chamber has a pressure law $P_v(\Delta V)$ with rest pressure P_0 :

$$\text{constant (input, vent): } P = P_0 \quad (\text{vent: } P_0 = 0), \quad (14)$$

$$\text{sealed liquid: } P = P_0 - K(\Delta V + V_{\text{ex}}), \quad K = \frac{100 s}{V_{\text{liq}}}, \quad V_{\text{ex}} = V_{\text{body}} - V_{\text{liq}}, \quad (15)$$

$$\text{sealed ideal gas: } P = (P_{\text{atm}} + P_0) \frac{V_g}{V_g + \Delta V} - P_{\text{atm}}. \quad (16)$$

For the liquid, the user gives a stiffness s in pressure per percent of volume change and a liquid volume V_{liq} , which defaults to the volume of the CAD body. If the chamber holds less liquid than its body volume ($V_{\text{ex}} > 0$), the liquid is in suction and pulls the membranes in; if it holds more, it inflates them. Water corresponds to $s \approx 22\,000$ kPa/%. Above about 1000 kPa/% the results hardly change, but the Newton iteration gets slower.

For the gas, Boyle’s law is applied to the gas share only: $V_g = (1 - f)(V_{\text{body}} + V_{\text{ghost}})$, where f is the liquid fraction (at most 99%) and V_{ghost} is an optional extra gas volume outside the modelled chamber, such as tubing. Any liquid in the chamber is incompressible, so every volume change goes into the gas. A state with $V_g + \Delta V \leq 0$ gets infinite energy and is rejected by the line search. A user supplied, differentiable law $P(\Delta V; P_0)$ is also supported.

5.3 Potential, load and tangent

The fluid’s work is $\Phi_v(\Delta V) = \int_0^{\Delta V} P_v(s) ds$. In closed form,

$$\Phi^{\text{liq}} = P_0 \Delta V - K \left(V_{\text{ex}} \Delta V + \frac{1}{2} \Delta V^2 \right), \quad \Phi^{\text{gas}} = (P_{\text{atm}} + P_0) V_g \ln \frac{V_g + \Delta V}{V_g} - P_{\text{atm}} \Delta V, \quad (17)$$

and a custom law is integrated with 16-point Gauss–Legendre quadrature. Differentiating $-\Phi_v(\Delta V_v(\mathbf{u}))$ gives the pressure load and its tangent:

$$\frac{\partial(-\Phi_v)}{\partial \mathbf{u}} = -P_v \mathbf{g}_v, \quad \frac{\partial^2(-\Phi_v)}{\partial \mathbf{u}^2} = -P_v \frac{\partial^2 \Delta V_v}{\partial \mathbf{u}^2} - P'_v \mathbf{g}_v \mathbf{g}_v^\top. \quad (18)$$

The force $P_v \mathbf{g}_v$ is exactly the follower pressure load, which acts normal to the *deformed* surface. The first tangent term is the usual load-stiffness term, and it is symmetric because the load has a potential. The second term couples every node of a chamber to every other node. It is a dense rank-one matrix, and Section 8 handles it without forming it. Since $P'_v \leq 0$ for every sealed chamber, $c_v = -P'_v \geq 0$ is a volumetric stiffness.

5.4 Load path

A fresh solve ramps every chamber from zero pressure: the solver uses $\lambda P_v(\Delta V)$ (and likewise for the slope and potential). A warm-started solve (Section 12) starts from a converged state with rest pressure P_0^{old} and blends to the new rest pressure,

$$P_v^\lambda(\Delta V) = (1 - \lambda) P_v(\Delta V; P_0^{\text{old}}) + \lambda P_v(\Delta V; P_0). \quad (19)$$

A warm-started solve with a single load step goes to $\lambda = 1$ at once, i.e. Newton starts at the new pressures from the old (or a predicted) shape; the load stepping of Section 10 still halves the increment if that fails.

6 Contact with rigid bodies

6.1 Exact signed distance

A rigid body is a closed, outward-oriented triangle mesh. For a point $\mathbf{p}$, the closest point $\mathbf{q}$ on each candidate triangle is found with the region tests of Ericson [3], which also report whether $\mathbf{q}$ lies on a face, an edge or a vertex. The sign of the distance comes from the *angle-weighted pseudo-normal* $\mathbf{N}_q$ of that feature [1]: the face normal on a face, the normalised sum of the two adjacent face normals on an edge, and the sum of the incident face normals weighted by their angles at a vertex. For a closed mesh this sign is exact:

$$d(\mathbf{p}) = \text{sign}((\mathbf{p} - \mathbf{q}) \cdot \mathbf{N}_q) \|\mathbf{p} - \mathbf{q}\|, \quad \nabla d = \text{sign}(\cdot) \frac{\mathbf{p} - \mathbf{q}}{\|\mathbf{p} - \mathbf{q}\|}. \quad (20)$$

$d < 0$ inside the solid. An *inverted* obstacle (a container whose inside is the free space) flips the sign, and several obstacles combine by taking the minimum. The Hessian of the distance is needed for a consistent tangent:

$$\nabla^2 d = \text{sign}(\cdot) \frac{1}{|d|} \begin{cases} \mathbf{0}, & \mathbf{q} \text{ on a face,} \\ \mathbf{I} - \hat{\mathbf{u}}\hat{\mathbf{u}}^\top - \hat{\mathbf{t}}\hat{\mathbf{t}}^\top, & \mathbf{q} \text{ on an edge with direction } \hat{\mathbf{t}}, \\ \mathbf{I} - \hat{\mathbf{u}}\hat{\mathbf{u}}^\top, & \mathbf{q} \text{ at a vertex,} \end{cases} \quad (21)$$

with $\hat{\mathbf{u}} = (\mathbf{p} - \mathbf{q})/\|\mathbf{p} - \mathbf{q}\|$. On a face the Hessian is zero because the face is flat.

Broad phase. A nearest-vertex distance gives an upper bound on the closest distance, and triangles whose bounding box is farther than that bound are skipped. Between Newton iterations the candidate triangles of each node are cached in Verlet lists [17]. At a rebuild, each node keeps every triangle within its current distance plus $2\delta_s$ (the skin δ_s is the larger of the contact reach and 1 % of the model size), and the list is rebuilt as soon as any node has moved more than δ_s . Until then the true closest triangle is guaranteed to be in the list, so the cached query gives *exactly* the result of a full search. A test checks this (`test_cached_contact_queries_match_the_full_search`).

6.2 Penalty energy

Each free node a of a shell keeps a clearance r_a from the obstacles: half the shell thickness, reduced to the rest distance where the CAD geometry already starts closer than that. The contact energy is a one-sided quadratic penalty,

$$E_{\text{rigid}} = \sum_a \frac{1}{2} k A_a \min(d(\mathbf{x}_a) - r_a, 0)^2, \quad (22)$$

with A_a the lumped nodal area (one third of the adjacent rest triangle areas). Weighting by area makes k a pressure per unit penetration, independent of the mesh. For an active node with gap $\gamma_a = d - r_a < 0$, the force and tangent are

$$\frac{\partial E}{\partial \mathbf{x}_a} = k A_a \gamma_a \nabla d, \quad \frac{\partial^2 E}{\partial \mathbf{x}_a^2} = k A_a \left(\nabla d \nabla d^\top + \gamma_a \nabla^2 d \right). \quad (23)$$

Choice of k . A penalty always allows some penetration. At a contact pressure p the penetration is about p/k . The application chooses

$$k = \frac{\max(p_{\text{ref}}, 1 \text{ kPa})}{0.05 t_{\text{min}}}, \quad (24)$$

where p_{ref} is the highest chamber pressure in the solve or sweep and t_{min} is the thinnest part, so the penetration stays around 5 % of the thinnest thickness.

Clamp walls. A membrane in CAD is usually clamped between rigid frame parts, so the free nodes next to its fixed boundary start right at a rigid wall. These nodes sit on the on/off kink of the penalty, and they made the Newton iteration cycle. Free nodes that share a triangle with a fixed node and start within $1.25 r_a$ of an obstacle are therefore excluded from rigid contact.

7 Contact between membranes

Membranes can also touch each other (not themselves). Each sheet is a mid-surface with thickness t , so node a of sheet A touches sheet B when it comes closer than $h = \frac{1}{2}t_A + \frac{1}{2}t_B$ to B 's mid-surface. As for rigid contact, h is reduced to the rest distance where the sheets already

start closer, for example where they meet at a frame. For every node–triangle pair within range, with d the distance from the node to the closest point of that triangle and penetration $\pi = h - d$,

$$E_{\text{sheet}} = \sum_{\text{pairs}} k A_a \varphi(\pi), \quad \varphi(\pi) = \begin{cases} 0, & \pi \leq 0, \\ \frac{\pi^3}{6\delta}, & 0 < \pi < \delta, \\ \frac{\pi^2}{2} - \frac{\delta\pi}{2} + \frac{\delta^2}{6}, & \pi \geq \delta, \end{cases} \quad \delta = 0.25 h. \quad (25)$$

The function φ is the ordinary quadratic penalty with its stiffness ramped up over the first δ of penetration. φ , φ' and φ'' are continuous at $\pi = 0$ and at $\pi = \delta$ (the three values at δ are $\delta^2/6$, $\delta/2$ and 1 from either side), so a pair coming into contact does not make the tangent jump. *All* triangles within range contribute, not only the closest. With the closest only, the force jumps from one triangle’s vertices to its neighbour’s when a node slides across an edge, and Newton cycled. Both directions (A ’s nodes on B and B ’s nodes on A) are included. The gradient and Hessian of each pair’s energy with respect to its twelve coordinates are exact (Section 8).

No tunnelling. A penalty can be stepped over: one large step can carry a node straight through the other sheet to where the penalty is zero again. As in the continuous collision handling of Incremental Potential Contact [10], every line-search step is therefore bounded conservatively. A node at distance d from a triangle can only reach it if the two approach each other by d , and in a step $\alpha \Delta \mathbf{u}$ they approach by at most $\alpha(\|\Delta \mathbf{x}_a\| + \max_{b \in \text{tri}} \|\Delta \mathbf{x}_b\|)$. The step fraction is limited to

$$\alpha \leq \min_{\text{pairs}} \frac{0.9 d}{\|\Delta \mathbf{x}_a\| + \max_b \|\Delta \mathbf{x}_b\|}, \quad (26)$$

so no pair closes more than 90 % of its current distance in one step. Pairs whose bounding-box distance already satisfies the bound are skipped without computing the exact distance.

8 Assembly and exact derivatives

Every energy above is written as a function of one element’s coordinates: a membrane triangle (9 values), a bending stencil (the triangle, its three opposite nodes and three edge rotations: 21 values), a pressure face (9), or a sheet-contact pair (12). The residual and tangent are the *exact* gradient and Hessian of the discrete energy, which gives quadratic Newton convergence. For the bending stencils, contact pairs and cap fans they are obtained by automatic differentiation with `torch.func` (`grad`, `hessian`, vectorised over the elements with `vmap`) [14].

The two most frequent elements, the membrane triangle and the pressure face, are differentiated by hand, because the overhead of automatic differentiation made them most of the cost of an evaluation. With $\mathbf{G} = [\mathbf{g}_0; \mathbf{g}_1; \mathbf{g}_2] \in \mathbb{R}^{3 \times 2}$ the rows $\mathbf{g}_1, \mathbf{g}_2$ of $\mathbf{D}_m^{-1}$ and $\mathbf{g}_0 = -\mathbf{g}_1 - \mathbf{g}_2$, the columns of $\mathbf{F}$ are $\mathbf{f}_\alpha = \sum_a G_{a\alpha} \mathbf{x}_a$, and both membrane energies are functions of $\mathbf{s} = (a, b, c) = (\mathbf{f}_0 \cdot \mathbf{f}_0, \mathbf{f}_1 \cdot \mathbf{f}_1, \mathbf{f}_0 \cdot \mathbf{f}_1)$, i.e. of $\mathbf{C} = \begin{bmatrix} a & c \\ c & b \end{bmatrix}$, with $\det \mathbf{C} = ab - c^2$ and $A = A_0 \sqrt{\det \mathbf{C}}$. The chain rule gives

$$\frac{\partial U}{\partial \mathbf{f}} = \sum_s U_{,s} \frac{\partial s}{\partial \mathbf{f}}, \quad \frac{\partial^2 U}{\partial \mathbf{f}^2} = \sum_s U_{,s} \frac{\partial^2 s}{\partial \mathbf{f}^2} + \sum_{s,r} U_{,sr} \frac{\partial s}{\partial \mathbf{f}} \otimes \frac{\partial r}{\partial \mathbf{f}}, \quad (27)$$

with $\partial a / \partial \mathbf{f} = (2\mathbf{f}_0, \mathbf{0})$, $\partial b / \partial \mathbf{f} = (\mathbf{0}, 2\mathbf{f}_1)$, $\partial c / \partial \mathbf{f} = (\mathbf{f}_1, \mathbf{f}_0)$ and constant second derivatives, and it is mapped to the nodes by $\partial \mathbf{f}_\alpha / \partial \mathbf{x}_a = G_{a\alpha} \mathbf{I}$. For a pressure face $V = \frac{1}{6} \mathbf{x}_0 \cdot (\mathbf{x}_1 \times \mathbf{x}_2)$,

$$\frac{\partial V}{\partial \mathbf{x}_0} = \frac{1}{6} \mathbf{x}_1 \times \mathbf{x}_2 \text{ (and cyclic),} \quad \left[\frac{\partial^2 V}{\partial \mathbf{x}_p \partial \mathbf{x}_{p+1}} \right]_{ij} = \frac{1}{6} \varepsilon_{ijk} x_{p+2,k}, \quad (28)$$

the diagonal blocks being zero and the Hessian symmetric. Tests check both against `torch.func` to round-off. On the user’s test neuron (2848 unknowns) this halved the cost of an evaluation; a 9×5 characterisation grid (Section 14) went from 88s to 65s with identical iterations. A further test compares the assembled residual and tangent with finite differences of Π (`test_residual_is_energy_gradient_and_tangent_is_its_jacobian`). Everything is computed in double precision.

The element contributions are assembled into a sparse matrix, and the rows and columns of fixed unknowns are dropped. The chamber terms $P'_v \mathbf{g}_v \mathbf{g}_v^\top$ of (18) are kept apart. Writing $\mathbf{U} = [\mathbf{g}_1, \dots, \mathbf{g}_m]$ for the volume gradients of the m chambers with $P'_v \neq 0$ and $\mathbf{C} = \text{diag}(c_v)$, the tangent is

$$\mathbf{K}_{\text{full}} = \mathbf{A} + \mathbf{U} \mathbf{C} \mathbf{U}^\top, \quad (29)$$

where $\mathbf{A}$ is sparse. Linear systems are solved with a sparse LU factorisation of $\mathbf{A}$ (SuperLU through SciPy [18]) and the Sherman–Morrison–Woodbury identity [7]:

$$\mathbf{K}_{\text{full}}^{-1} \mathbf{b} = \mathbf{y} - \mathbf{Z} \left(\mathbf{C}^{-1} + \mathbf{U}^\top \mathbf{Z} \right)^{-1} \mathbf{U}^\top \mathbf{y}, \quad \mathbf{y} = \mathbf{A}^{-1} \mathbf{b}, \quad \mathbf{Z} = \mathbf{A}^{-1} \mathbf{U}. \quad (30)$$

This needs only m extra back-substitutions and one small $m \times m$ solve.

9 Newton iteration

At a fixed load factor λ , the solver iterates from the current state until

$$\|\mathbf{R}\| \leq \tau_a + \tau_r \max(\|\mathbf{f}_{\text{int}}\|, \|\mathbf{f}_{\text{pres}}\|, \|\mathbf{f}_{\text{cont}}\|), \quad (31)$$

with defaults $\tau_r = 10^{-8}$ and $\tau_a = 10^{-12}$. The reference is the largest of the internal, pressure and contact force norms (free unknowns only). An iteration also counts as converged when the undamped Newton step is below 10^{-10} times the model size (round-off floor). Contact energies are only C^1 where a closest point crosses a triangle edge, which puts a floor under the attainable residual. So when the residual has not halved for three iterations and is within 1000 times the tolerance, the iteration also stops as converged.

Each iteration tries the following steps in order, until one decreases the energy:

1. **Newton step with second-order correction.** Solve $\mathbf{K}_{\text{full}} \mathbf{s} = -\mathbf{R}$ with (30) and search along $\mathbf{s}$, correcting every trial point as described below.
2. **Levenberg–Marquardt steps.** If the Newton step fails, for example because the tangent is indefinite or singular (a slack membrane, or a limit point), damp it: $(\mathbf{A} + \mu \bar{a} \mathbf{I} + \mathbf{U} \mathbf{C} \mathbf{U}^\top) \mathbf{s} = -\mathbf{R}$, with $\bar{a}$ the mean absolute diagonal of $\mathbf{A}$. After a failure $\mu \leftarrow \max(10\mu, 10^{-6})$, for up to four attempts. When the first attempt of an iteration is accepted with a full step ($\alpha = 1$), μ is reduced tenfold, or set to zero below 10^{-9} . μ carries over between iterations and load steps, so the first attempt is the undamped Newton step only once the damping has died out.
3. **Gradient step.** As a last resort, search along the Jacobi-preconditioned steepest descent direction $\mathbf{s} = -\mathbf{R} \oslash (|\text{diag } \mathbf{A}| + (\mathbf{U} \circ \mathbf{U}) |\mathbf{c}| + 10^{-8} \bar{a})$, with up to 40 backtracking steps. This always decreases the energy for a small enough step. If it also fails, the Newton solve reports failure.

9.1 Line search on the energy

Every candidate step is first scaled so that no coordinate moves more than 10% of the model size, and it is limited by the no-tunnelling bound of Section 7. A step length α is then accepted under the Armijo condition [13]

$$\Pi(\mathbf{u}_0 + \alpha \mathbf{s}) \leq \Pi(\mathbf{u}_0) + 10^{-4} \alpha \mathbf{R}^\top \mathbf{s}. \quad (32)$$

If the condition fails, the next α is the minimiser of the quadratic through $\Pi(\mathbf{u}_0)$, the slope $\mathbf{R}^\top \mathbf{s}$ and $\Pi(\mathbf{u}_0 + \alpha \mathbf{s})$, kept within $[0.1\alpha, 0.5\alpha]$. It is $\alpha/10$ when the trial energy is infinite (a gas pocket squeezed to zero). A step whose energy change is within round-off of zero is also accepted if it reduces the residual. **The energy is the only merit function.** Mixing in a residual-decrease criterion made the iteration cycle as contact nodes switched in and out of contact.

9.2 Second-order volume correction

Stiff sealed chambers need special treatment. A chamber’s volume is a nonlinear function of the node positions, so along a straight Newton step the actual volume change differs from the linearised one by $O(\alpha^2 \|\mathbf{s}\|^2)$. A nearly incompressible chamber (large c_v) turns this small volume error into a large energy $\frac{1}{2}c_v e_v^2$. The line search then rejects good Newton steps and convergence slows to a crawl. This is the Maratos effect known from constrained optimisation, and the standard remedy is a second-order correction [13]. Each trial point $\mathbf{u}_0 + \alpha \mathbf{s}$ is moved back onto the linearised volumes. With the volume error

$$e_v = \Delta V_v(\mathbf{u}_0 + \alpha \mathbf{s}) - (\Delta V_v(\mathbf{u}_0) + \alpha \mathbf{g}_v^\top \mathbf{s}), \quad (33)$$

the correction is the smallest change, in the norm of $\mathbf{A}$, that removes it to first order:

$$\min_{\boldsymbol{\delta}} \frac{1}{2} \boldsymbol{\delta}^\top \mathbf{A} \boldsymbol{\delta} \quad \text{s.t.} \quad \mathbf{U}^\top \boldsymbol{\delta} = -\mathbf{e} \quad \implies \quad \boldsymbol{\delta} = -\mathbf{Z} \left(\mathbf{U}^\top \mathbf{Z} \right)^{-1} \mathbf{e}, \quad (34)$$

where $\mathbf{Z} = \mathbf{A}^{-1} \mathbf{U}$ is already available from (30). With the correction, stiff and soft chambers converge in a similar number of iterations.

10 Load stepping

The pressures are applied in load steps $\lambda = 0 \rightarrow 1$, with initial increment $1/N$ ($N = 10$ by default). After each converged step the state is stored; the application can play these states back. The increment

- grows by a factor 1.5 (up to $1/N$) when a step converged in at most four iterations;
- is halved, and the step retried from the last converged state, when Newton fails;
- is considered too small below 10^{-3} , at which point the solve stops and reports the last converged load factor.

A snap-through, such as a membrane ballooning past its limit point, has no nearby equilibrium, so cutting the increment does not help. The first time the increment falls below $0.25/N$, the solver makes one long “relaxation” attempt at the target load with five times the usual iteration limit. The energy-descent iteration of Section 9 can then carry the structure to its new equilibrium.

11 From CAD to model

Import. The STEP file is read with gmsh’s OpenCASCADE kernel [4], in millimetres. Every solid is one body. Names come from the product labels, or, for multi-body parts from Fusion 360 whose names gmsh does not read, from the MANIFOLD_SOLID_BREP records of the file.

Meshing. Every body’s boundary is triangulated with gmsh (Frontal–Delaunay, size set at the CAD points). For membranes and shells the element size is the shortest *in-plane* side of the bounding box divided by a number of elements per side (default 10). For solids it is the shortest side divided by that number, but never finer than the diagonal divided by three times that number, so that thin rigid parts do not produce huge meshes. Each closed surface mesh is

then oriented outward: the faces of each connected component are made consistent by walking across shared edges, the component is flipped if its signed volume is negative, and a component nested inside an odd number of others is flipped again, because it bounds a cavity. The nesting depth is found with the generalised winding number (below).

Mid-surfaces. A membrane or shell body is replaced by its mid-surface. The body’s largest CAD face is taken as the primary face. From each of its nodes a ray is cast inward along the vertex normal (Möller–Trumbore [11]) to the opposite faces, which measures the local thickness d . The thickness is the median of these distances. Local values outside $[\text{median}/3, 3 \text{ median}]$, or rays that miss, are replaced by the median, and every node is moved inward by $\frac{1}{2}d$. The mid-surface keeps the orientation of the primary face.

Boundary conditions. By default all boundary nodes of the mid-surface are fixed (with clamped rotations for shells). Alternatively, only boundary nodes within $0.75t$ of a rigid body are fixed, which models a membrane clamped by a frame on some edges and free elsewhere.

Chamber coupling. The software must find out which chambers act on which side of which membrane. From the centroid of every mid-surface triangle, a probe point is placed on each side at distance $\frac{1}{2}t + \max(\frac{1}{4}t, 10^{-3})$ along the normal. A probe lies inside a chamber when the generalised winding number of the chamber’s mesh exceeds $\frac{1}{2}$ [9]. The winding number is evaluated as a sum of triangle solid angles [16]:

$$w(\mathbf{p}) = \frac{1}{4\pi} \sum_{\text{tri}} 2 \text{atan2} \left(\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}), abc + (\mathbf{a} \cdot \mathbf{b})c + (\mathbf{a} \cdot \mathbf{c})b + (\mathbf{b} \cdot \mathbf{c})a \right), \quad (35)$$

with $\mathbf{a}, \mathbf{b}, \mathbf{c}$ the triangle vertices relative to $\mathbf{p}$ and a, b, c their lengths. If at least 30 % of the probes on one side are inside, the chamber is coupled to that side of the membrane with the corresponding σ of (13). A chamber detected on both sides has no net effect and is ignored with a warning. A warning is also given when the coverage is below 90 %, because the pressure is then applied to the whole membrane.

Chamber parameters. Constant-pressure chambers are the inputs. Sealed liquid chambers use (15), with V_{body} the CAD volume. Sealed gas chambers use (16) with $P_{\text{atm}} = 101.325 \text{ kPa}$. Vents are constant-pressure chambers at zero.

12 Parameter sweeps

A sweep solves the model on a grid of one or two input-chamber pressures. The first point is a fresh solve. Every following point is warm-started from the previous solution, ramping only the change in rest pressure, with two load steps. If that fails, the point is solved fresh. One penalty stiffness, chosen for the highest pressure in the sweep, is used for all points, so that neighbouring points are consistent. Warm starting is several times faster, because contact is already established. A test checks that it reaches the same state as a fresh solve (`test_warm_start_reaches_the_same_state_as_a_fresh_solve`). The characterisation of a full neuron (Section 14) goes further: one load step and a predicted starting state.

13 The activation-function device

The second space of the software simulates the output stage of a neuron: a valve in which the pressure difference Δp across a membrane squeezes a soft tube (the *channel*) against a fixed rigid body, through a part bonded to the membrane (a *pusher*, rigid or deformable). Gas flows

through the tube from a supply to a sink. The quantity of interest is how the pressure divides along the tube, and with it the mass flow, as the tube is clamped: the squeezed tube is a variable flow resistance in series with the rest of the gas path, a pressure divider. The result, the tube's smallest cross-section A , the mass flow $\dot{m}$ and the pressures along the tube as functions of Δp , is saved with the device as a design file, so that it can be used as a part without simulating it again. In the neuron, Δp is the *pre-activation* (the pressure difference set by the pre-activation chamber, section 12) and the device maps it to the *activation*: the gas pressure of a segment of the tube, chosen and named by the user; a device can have several such outputs (section 13.10). The software's two spaces are named accordingly: *inputs to pre-activation* (the neuron) and *pre-activation to activation* (this device). The model reuses the membranes, fluid chambers, rigid contact and Newton solver of the previous sections and adds a volumetric solid, moving rigid bodies, chamber walls on a solid, contact of a solid with itself, the cross-section measurement and a steady flow network.

13.1 Roles

Besides Membrane/Shell, a device has one *Channel* (the tube) and *Fluid* bodies: gas regions at a *constant* pressure (a supply or a sink) or with a *dynamic* pressure, which follows from the flow. The dynamic fluid lining the tube is cut into N segments along the tube. A *Rigid body* is either fixed or free (six rigid-body unknowns), and a *Solid* is a deformable part meshed like the tube, free or fixed where it touches fixed rigid bodies. A membrane is bonded to every free rigid body or solid whose surface its face touches; that part (e.g. a pusher) carries the membrane's load to the tube. Every tube face belongs to the fluid whose mesh contains a probe point placed just inside the tube from it (generalised winding number, as in the chamber coupling of Section 11). The channel axis $\mathbf{a}$ is the long principal direction of the fluid lining the tube, oriented from its higher-pressure end. The tube is held at its end faces: the planar CAD faces of the tube, normal to $\mathbf{a}$, that do not line the channel. Δp acts on the membrane as a constant-pressure chamber on the face that points away from the tube, so that positive Δp closes it; the closing direction runs from the membranes' area-weighted centroid to the channel axis.

13.2 Solid tube

The tube is a volumetric solid meshed with 10-node (quadratic) tetrahedra. A tube is closed by bending its walls, and linear tetrahedra with one or two elements through a wall are far too stiff in bending; quadratic elements are not. The material is the compressible neo-Hookean solid [2]

$$W(\mathbf{F}) = \frac{\mu}{2} \left(\text{tr } \mathbf{F}^T \mathbf{F} - 3 - 2 \ln J \right) + \frac{\lambda}{2} (\ln J)^2, \quad J = \det \mathbf{F}, \quad (36)$$

with μ and λ from E and ν , integrated with the 4-point rule of the tetrahedron. $\mathbf{F} = \sum_a \mathbf{x}_a \otimes \nabla_{\mathbf{X}} N_a$ with the quadratic shape functions on the (curved) rest geometry, so the energy of a homogeneous deformation is exact. J is computed as the triple product of the columns of $\mathbf{F}$: the second derivative of a general determinant routine was not finite at $\mathbf{F} = \mathbf{I}$. The element energy, gradient and Hessian come from automatic differentiation as for the shells (Section 8). Each 6-node boundary triangle is split into four flat triangles through its mid-edge nodes; these are the faces used for contact, chamber walls, cross-sections and display.

13.3 Free rigid bodies

A free rigid body, e.g. a rigid pusher, has six unknowns $\mathbf{q} = (\mathbf{t}, \boldsymbol{\theta})$: a body point $\mathbf{X}$ moves to

$$\mathbf{x}(\mathbf{X}) = \mathbf{c}_0 + \mathbf{t} + \mathbf{R}(\boldsymbol{\theta})(\mathbf{X} - \mathbf{c}_0), \quad (37)$$

with $\mathbf{c}_0$ the rest centroid and $\mathbf{R}$ Rodrigues' rotation, evaluated with Taylor series for small $|\boldsymbol{\theta}|$ so that it is smooth, with its derivatives, through $\boldsymbol{\theta} = \mathbf{0}$. The body stores no energy; it is moved by two couplings.

Bond to the membrane. Membrane nodes within $0.6t$ of the body’s surface at rest (the mid-surface lies $t/2$ above the face it is bonded to) are tied to the point of the body they started on,

$$\Pi_{\text{tie}} = \sum_i \frac{1}{2} k_{\text{tie}} A_i |\mathbf{x}_i - \mathbf{x}(\mathbf{X}_i)|^2, \quad k_{\text{tie}} = 10k, \quad (38)$$

a stiff penalty on the 9 unknowns $(\mathbf{x}_i, \mathbf{t}, \boldsymbol{\theta})$ of each tie.

Contact with the tube. The surface nodes of the tube and of every solid are kept outside the body’s closed surface with the penalty of Section 6, the signed distance being evaluated in the body frame, $d(\mathbf{x}) = d_0(\mathbf{c}_0 + \mathbf{R}^\top(\mathbf{x} - \mathbf{c}_0 - \mathbf{t}))$. For exact derivatives with respect to both the node and the body, d_0 is replaced by its second-order expansion around the current point $\mathbf{y}^*$ (value, gradient $\mathbf{n}$ and Hessian $\mathbf{H}$ from the exact distance query),

$$\tilde{d}(\mathbf{y}) = d_0(\mathbf{y}^*) + \mathbf{n} \cdot (\mathbf{y} - \mathbf{y}^*) + \frac{1}{2}(\mathbf{y} - \mathbf{y}^*)^\top \mathbf{H}(\mathbf{y} - \mathbf{y}^*), \quad (39)$$

which has the same value, gradient and Hessian at $\mathbf{y}^*$, and the energy $\frac{1}{2}kA_i \min(\tilde{d} - g_i, 0)^2$ is differentiated automatically. The offset $g_i = \min(0, d_i^{\text{rest}})$ makes the rest contact neutral: the tube and the pusher start touching, and the facets of their curved CAD faces overlap by a few μm . The same rule applies to the tube against fixed rigid bodies.

13.4 Deformable parts

A solid part (e.g. a soft pusher) is meshed and modelled like the tube. A membrane node bonded to it follows the material point of the solid’s surface it started on, $\Pi = \frac{1}{2}k_{\text{tie}}A_i|\mathbf{x}_i - \sum_k w_k \mathbf{x}_k - \mathbf{r}_0|^2$ with the barycentric weights w_k of its closest triangle and its rest offset $\mathbf{r}_0$ (not rotated with the surface, which is exact for the small rotations of a bonded face). Two solids (the part and the tube) touch through a node-to-surface penalty in both directions: each surface node of one is paired with its closest triangle of the other, and its signed distance is the plane distance when the closest point is inside the triangle, else the distance to the closest edge or vertex signed by the triangle’s normal; $\frac{1}{2}kA_i \min(d - g_i, 0)^2$ with the neutral rest offset g_i as above. The closest triangle can change from one evaluation to the next, so this energy is only continuous where a node crosses between triangles; on the test valve it converges, 2 to 4 times slower than a rigid pusher.

13.5 Chamber walls on a solid

The inside of the tube is not bounded by clamped membranes, so the cone volumes of Section 5 would depend on the apex: where the input and output halves meet, the wall has a boundary loop that moves. Each patch of wall (a set of oriented boundary triangles; the solid’s outward normals point into the channel, so its side is -1) is therefore closed by a fan of triangles from every boundary loop that has a free node to the loop’s centroid $\mathbf{c}_\ell = \frac{1}{m} \sum_k \mathbf{x}_{\ell k}$:

$$V = \sum_{f \in \text{wall}} V_{\text{cone}}(\mathbf{x}_f) + \sum_{\ell} \sum_k \frac{1}{6} \mathbf{x}_{\ell, k+1} \cdot (\mathbf{x}_{\ell k} \times \mathbf{c}_\ell), \quad (40)$$

the loop taken in the direction of the wall’s boundary half-edges. The volume of a closed surface is independent of the cone apex, and loops of fixed nodes (the tube ends) contribute a constant that drops out of ΔV . Each cap is one element over the $3m$ coordinates of its loop, with its gradient and Hessian from automatic differentiation. The cap carries the pressure difference of the two halves as an axial force on the nodes of the loop; in steady flow through a constriction that force is real, transmitted by viscous shear to the walls, and it vanishes when both halves are at the same pressure. Two neighbouring patches (e.g. two segments of the tube) that share a loop split the tube’s volume exactly.

13.6 The tube closing on itself

When the tube closes, its inner walls touch. The node-to-triangle penalty of Section 7 is used with explicit node and face sets on one body: the wall triangles on the membrane's side of the channel against those on the far side, and the reverse, with contact distance $h = 0.02 H$ (H the smallest inside dimension of the tube), reduced to the rest distance as before. A node lying on a triangle of the other set (on the line where the two sets meet) has zero rest distance and never becomes active; it is also left out of the step bound, which would otherwise allow no step at all.

13.7 Cross-sectional area

Sections are planes normal to $\mathbf{a}$ at n positions along the channel, placed in the rest configuration (a vertex on a plane counts as above it). Every wall triangle a plane cuts gives a segment between two material points on its edges, oriented by the triangle's normal. In the deformed configuration the loop encloses, projected on the plane,

$$A = \frac{1}{2} \sum_{\text{segments } (\mathbf{p}, \mathbf{q})} (\mathbf{p} \times \mathbf{q}) \cdot \mathbf{a}, \quad (41)$$

so the segments need not be ordered. Walls pressed through each other (the penalty's penetration) count negatively and the area is clipped at zero. The sections cut every tube face that lines a fluid, not only the dynamic fluid's, so that a section near the junction of two fluids keeps all its triangles. The reported A is the smallest section: the constriction that sets the flow. A_0 is the same measure at rest, so the faceting of curved walls affects A and A_0 alike. With the walls in contact the area does not reach zero: the contact gap h and the corners of the lumen leave a floor of roughly h times the width of the channel.

13.8 Flow network

Where two fluid bodies touch, and where a dynamic fluid faces the outside (a face whose probe lies in no body, checked again further out so that the thin gaps between the facets of two curved CAD faces do not count), there is a *flow connection*: an opening (no resistance), an orifice with a user-defined law, or closed. The gas is a network of pressure nodes: the $N + 1$ ends of the tube's segments, one node per other fluid (fixed for constant fluids) and the outside at 0 kPa. Its edges are the segments and the orifices, each with a law for the pressure drop at a positive mass flow,

$$\Delta p = f(\dot{m}, \rho, \mu, A, P, h, w, L, D_h, p, p_{\text{up}}, p_{\text{down}}, \rho_{\text{up}}), \quad (42)$$

in SI units, the flow following the sign of the pressure difference. The gas is ideal, $\rho = p_{\text{abs}}/(R_s T)$, taken at the mean pressure of the edge (ρ) or upstream (ρ_{up} , for orifices); R_s , T , the atmospheric pressure and the viscosity μ are settings. A segment's geometry is its smallest cross-section (area A , wetted perimeter P , height h along the closing direction, width w , $D_h = 4A/P$) and its length L ; an orifice's is its contact face. Defaults: laminar segments, $\Delta p = 32\mu L \dot{m}/(\rho A D_h^2)$, and a sharp-edged orifice, $\Delta p = (\dot{m}/(0.61A))^2/(2\rho_{\text{up}})$. Openings join nodes. The unknown pressures satisfy the steady mass balance $\sum \dot{m} = 0$ at every free node; each edge's flow for a given pressure difference comes from inverting its law (Brent's method on $|\dot{m}|$), and the balance is solved with a Powell hybrid method.

13.9 Structure–flow iteration

The flow sets the pressures on the tube wall (segment k at the mean of its two end nodes; the wall facing another fluid at that fluid's pressure, as constant-pressure chambers on the wall patches), the wall pressures change the tube's shape, and the shape changes the flow. At each Δp they are iterated: solve the structure (warm-started, ramping only the change of the pressures), measure

the segments, solve the network, update the wall pressures, under-relaxed when the change grows (the iteration can oscillate between an open and a shut tube), until no wall pressure changes by more than a tolerance (default 0.01 kPa). Contact and tie penalties are sized at the start of the sweep: $k = 4p_{\text{ref}}/(0.02H)$ with p_{ref} the largest of $|\Delta p|$ and the constant fluid pressures, so contact penetrations are about 1 % of H .

13.10 The outputs

The outputs of the device (activations) are named segments of the fluid inside the tube, set in that fluid's properties. Each is the gas pressure of its segment k , the same pressure that acts on the segment's wall,

$$p_{\text{act}}(\Delta p) = \frac{1}{2} (p_{k-1}(\Delta p) + p_k(\Delta p)), \quad (43)$$

with p_{k-1}, p_k the node pressures at the segment's ends; without any, the downstream segment is the one output. The node pressures of every simulated point are stored, so outputs can be added or changed after the sweep without simulating again; they are saved with the design.

13.11 A design as a part of a neuron

A simulated device is saved as a design file and used in a neuron without simulating it again. There its membrane is one wall of the pre-activation chamber (the part is given the role *Activation membrane* and the design file), and all the rest of the neuron needs from it is how much volume it sweeps at a pressure difference. The study records, at every Δp , the change V of the volume on the membrane's driving side (the Δp chamber of the device, with the flow's wall pressures acting on the tube); in the neuron the membrane is replaced by that curve. Its unknown is one mean deflection q (a length, so the step cap applies to it) with swept volume $v = A_m q$, A_m the area of the neuron's part, and it stores the energy

$$E(v) = \int_0^v \Delta p(s) ds, \quad (44)$$

with $\Delta p(v)$ the inverse of the simulated $V(\Delta p)$: a monotone cubic (PCHIP) through the simulated points, extended linearly with its end slopes, so E is C^2 inside the data and C^1 at its ends; points where V stops growing with Δp (a shut tube) are dropped, and without points below $\Delta p = 0$ the curve is mirrored about its $\Delta p = 0$ point. As a chamber wall it adds $+v$ to the change of volume of the *driving* chamber and $-v$ to that of a chamber on the tube side (the two sides are found as for a membrane, section 11; the driving one is chosen by the user or is the closed chamber the part touches), so the stationarity of the potential gives

$$\Delta p(v) = P_{\text{drive}} - P_{\text{tube}} \quad (45)$$

at equilibrium, with $P_{\text{tube}} = 0$ when no chamber is on the tube side. Its gradient and tangent with respect to q are $A_m \Delta p$ and $A_m^2 d\Delta p/dv$; its volume gradient enters the rank-one chamber terms like a membrane's. The device's outputs (every named p_{act} , smallest area, mass flow) are read from the design at the solved Δp by linear interpolation, and held at the end values outside the simulated range. There the design is extrapolated rather than interpolated (the swept volume continues linearly with the end slope), and the software warns whenever a solved Δp lies outside the simulated range. The design file also stores the full finite-element solution at every simulated Δp (the membranes' nodes, the tube's surface nodes and the rigid bodies' poses, in single precision), so the deformed device can be shown at any Δp by interpolating linearly between the neighbouring points, without simulating it again. The swept volume depends on the device alone, so the replacement is exact up to the interpolation, provided the device in the neuron sees the conditions it was simulated under (supply and sink pressures of its tube).

14 The full neuron and its characterisation

The *full neuron* workbench puts the two halves together: it imports a neuron project and a design, links the design to one of the neuron’s membranes as above, and solves and sweeps the result like the neuron. Only the fluids’ parameters (pressures, volumes, stiffnesses) can be changed there, not the kinds of parts or chambers and not the design, since those would need a new simulation. Any of the chamber pressures and volume changes, the pre-activation Δp and the design’s outputs can be recorded at every solve. The two models are shown side by side, each with its own position and rotation, which only affect the display.

Pinned components. Both components are embedded in the workbench file as they were imported, so later changes to their files do not leak in unnoticed (as with referenced components in a CAD assembly). The design is compared with its file by a hash of the file’s bytes; the neuron by its content (paths made absolute, display settings left out), so saving it unchanged keeps it current. An update of the neuron is a three-way merge with the version it was imported at as the base: a value changed only in the file is taken, a value changed only in the workbench (a fluid parameter or a solver setting) is kept, and a value changed in both is the user’s choice.

The workbench *characterises* the linked neuron: any number of chamber parameters $\theta_1, \dots, \theta_m$ (pressures, ghost volumes, liquid shares, stiffnesses, fluid volumes) are given values $\theta_k \in \{\theta_k^{(1)}, \dots, \theta_k^{(n_k)}\}$ and the neuron is solved at every one of the $\prod_k n_k$ combinations. A changed volume law (gas share, stiffness, liquid volume) is applied to the built model without rebuilding it.

Order and starting guesses. The points are visited in serpentine (boustrophedon) order: the last axis runs forwards and backwards alternately inside each step of the axis before it, so consecutive points differ by one step of one parameter. Each point is solved in a single load step (Section 5) from a predicted state. If the point b before the previous point a lies on the same axis k , the unknowns of all bodies (node coordinates, edge rotations, rigid poses, empirical deflections) are extrapolated along the line through the two solutions,

$$\mathbf{u}_{\text{guess}} = \mathbf{u}_a + t(\mathbf{u}_a - \mathbf{u}_b), \quad t = \frac{\theta_k^{\text{new}} - \theta_k^{(a)}}{\theta_k^{(a)} - \theta_k^{(b)}}, \quad (46)$$

otherwise the previous solution $\mathbf{u}_a$ is the guess. If Newton fails from the guess it is repeated from $\mathbf{u}_a$, and only then from rest with the full load stepping. On the user’s test neuron (about 3.4k unknowns, 0.3s per Newton iteration) dropping the two-step ramp saved 20 %, and the prediction another 20 % on a 9×5 pressure grid (111s $\rightarrow$ 88s). The cost is set by the first few kPa, where the membrane is still slack and comes into contact (10–25 iterations per point at any step size); beyond that a point takes 2–3 iterations, so a dense axis is cheap: 16 points over 0–20 kPa take three times as long as one 20 kPa step. Choosing between the guess and $\mathbf{u}_a$ by the smaller residual was tried and was slower: the residual misjudges which start is closer.

The dataset. Every recorded quantity y is stored on the grid, $y_{i_1 \dots i_m}$, together with whether the point converged and whether the design was extrapolated there, and optionally the deformed sheets at every point. The dataset is saved with the workbench file, with a fingerprint of everything it depends on (the neuron’s fixed parameters, the solver settings, the link and the design file), so a model changed afterwards is flagged. It stands in for the neuron wherever it is used later: a quantity at any θ inside the grid is the multilinear interpolation of y over the enclosing grid cell,

$$y(\theta) = \sum_{c \in \{0,1\}^m} y_{i_1+c_1, \dots, i_m+c_m} \prod_{k=1}^m (c_k w_k + (1-c_k)(1-w_k)), \quad w_k = \frac{\theta_k - \theta_k^{(i_k)}}{\theta_k^{(i_{k+1})} - \theta_k^{(i_k)}}, \quad (47)$$

and a θ outside the grid is extrapolated linearly and reported as out of range. A run that is stopped early is kept and marked incomplete; its missing points are empty.

Display. Both models are drawn at the selected grid point, coloured by the magnitude of the displacement on one common scale: the neuron’s sheets from the stored shapes, the design’s bodies from its stored solution interpolated at that point’s Δp (Section 13.11). A point whose Δp is outside the design’s simulated range is marked as extrapolating in the table and the view.

Analysis. The *Analysis* space slices the stored grid for plotting: one recorded quantity against one or two swept parameters, with up to three more parameters laid out as columns, rows and layers of panels at a chosen number of their values, and every other parameter held at one value. Held values are either snapped to the nearest simulated value or interpolated with the multilinear formula above restricted to the held axes, clamped to the grid (no extrapolation); a corner that was not solved makes the value undefined, drawn as a gap. The deformed shapes at a chosen point are interpolated with the same weights.

15 Verification

The repository contains an automated test suite (`python -m pytest tests`). Grouped by what they establish, the tests check that:

- **the discrete system is consistent:** the residual is the gradient of the energy and the tangent is its Jacobian, by finite differences, with active contact, for gas chambers, and for sheet contact;
- **the elements converge to known solutions:** clamped square and circular plates against Kirchhoff plate theory [15]; an inflated circular membrane against Hencky’s solution [8]; bending energy independent of mesh orientation;
- **the chambers behave as intended:** the volume change of an imposed paraboloid; a very stiff sealed chamber gives the same state as a constant-pressure chamber at the same final pressure; a custom pressure law matches the equivalent built-in law; Boyle’s law on the gas share; a gas pocket cannot be squeezed to zero; an under-filled liquid chamber pulls the membrane to its liquid volume;
- **contact is correct:** signed distances match a brute-force search on convex and non-convex shapes; cached queries match the full search; a membrane inflates against a flat obstacle; an inflating sheet lifts another without passing through it;
- **the solver is reproducible:** a warm-started solve reaches the same state as a fresh one;
- **the CAD pipeline works:** part names, closed outward meshes, mid-surfaces, measured thickness, chamber–membrane coupling and project save/load on a generated test assembly;
- **the activation-function model is consistent:** the solid’s energy of a homogeneous deformation is exact for linear and quadratic tetrahedra, and its Hessian is finite at rest and matches finite differences; the rotation is smooth through zero; the energy, residual and tangent of the assembled device (tube, membrane, pusher with its bond and contact, capped chamber walls) agree with finite differences to 10^{-9} , including the six rigid-body unknowns; two capped halves split a channel’s volume exactly; the section area of a polygonal tube and of its flattened shape is exact; the flow network reproduces a linear pressure divider exactly and an orifice’s flow matches its law by hand; on a generated round-tube valve the flow connections are found without false openings, the pressure falls along the tube, squeezing reduces the mass flow, and the design file round-trips;

- **a design replaces its membrane:** the energy, pressure difference and stiffness of the empirical wall agree with finite differences inside and outside the data; driven by a constant pressure it sweeps the simulated volume; a sealed gas chamber closed by a membrane and by the membrane's own empirical curve reach the same pressure (within 0.05 kPa); in a neuron the solved Δp is the driving minus the tube-side pressure, the design's outputs follow it in a sweep; the stored solution of a design is interpolated between its points, and a pre-activation outside its range raises the extrapolation warning;
- **the full neuron and its characterisation work:** linking replaces the chosen membrane without changing the imported neuron; the workbench file round-trips; the serpentine order moves one step of one axis at a time; the prediction lies on the line through the last two solutions; a grid over two pressures and a chamber stiffness converges everywhere, a stiffer chamber moves less, the dataset is saved, reloaded, interpolated (exact at grid points, bounded between them) and flagged as outdated when a fixed parameter changes but not a swept one; an interrupted run is kept as incomplete.

16 Limitations

- **Static only.** There is no inertia, damping or fluid flow, so the switching dynamics of the output stage and the flow through resistors are outside the model.
- **Lumped chambers.** Every chamber has one uniform pressure. Where a chamber covers only part of a membrane, the pressure is still applied to the whole membrane (with a warning).
- **Penalty contact.** Contact allows a small penetration (about 5 % of the thinnest thickness by default) and makes the tangent stiff. There is no friction and no self-contact of a sheet. Sheet contact can still stall under very large pushes (one test of this case does not yet pass).
- **Nearly incompressible chambers** use a stiff linear pressure law rather than a mixed formulation with the pressure as an unknown. Very high stiffness is therefore ill-conditioned, although the second-order correction keeps it usable.
- **Material models.** Only incompressible neo-Hookean and St. Venant–Kirchhoff sheets are available, with no viscoelasticity or damage, and there is no check that stretches stay within the material's limits.
- **Activation-function device.** The tube is a compressible neo-Hookean solid. Its displacement-based tetrahedra stiffen (lock) as $\nu \rightarrow 0.5$; use $\nu \leq 0.49$. The area of a closed tube does not reach zero (contact gap and lumen corners, about 2 % of A_0 on the test device). The flow is steady, isothermal and lumped (one pressure per segment); choked or compressible orifice flow is modelled only by a user-written law. Squeezing far past closure makes the solve slow (tens of seconds per point). A design used in a neuron is only valid for the supply and sink pressures it was simulated with, and only its swept volume couples back to the neuron: the tube's flow does not reach the neuron's chambers.
- **Characterisation.** The dataset is a full grid, so its size is the product of the numbers of values (n^m points for m parameters); between grid points it is linear, so strongly curved responses need dense axes there. It is only valid for the fixed parameters it was run with (checked by its fingerprint).
- **Robustness.** The globalised Newton method has no convergence guarantee. Snap-through and hard contact cases can still stall before full load; the solve then reports the last converged load factor rather than a result at full load.

References

- [1] J. A. Bærentzen and H. Aanæs, “Signed distance computation using the angle weighted pseudonormal,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 11, no. 3, pp. 243–253, 2005.
- [2] J. Bonet and R. D. Wood, *Nonlinear Continuum Mechanics for Finite Element Analysis*, 2nd ed. Cambridge University Press, 2008.
- [3] C. Ericson, *Real-Time Collision Detection*. Morgan Kaufmann, 2004.
- [4] C. Geuzaine and J.-F. Remacle, “Gmsh: a 3-D finite element mesh generator with built-in pre- and post-processing facilities,” *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009.
- [5] E. Grinspun, A. N. Hirani, M. Desbrun, and P. Schröder, “Discrete shells,” in *Proc. ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2003, pp. 62–67.
- [6] E. Grinspun, Y. Gingold, J. Reisman, and D. Zorin, “Computing discrete shape operators on general meshes,” *Computer Graphics Forum*, vol. 25, no. 3, pp. 547–556, 2006.
- [7] W. W. Hager, “Updating the inverse of a matrix,” *SIAM Review*, vol. 31, no. 2, pp. 221–239, 1989.
- [8] H. Hencky, “Über den Spannungszustand in kreisrunden Platten mit verschwindender Biegesteifigkeit,” *Zeitschrift für Mathematik und Physik*, vol. 63, pp. 311–317, 1915.
- [9] A. Jacobson, L. Kavan, and O. Sorkine-Hornung, “Robust inside-outside segmentation using generalized winding numbers,” *ACM Transactions on Graphics*, vol. 32, no. 4, 2013.
- [10] M. Li, Z. Ferguson, T. Schneider, T. Langlois, D. Zorin, D. Panozzo, C. Jiang, and D. M. Kaufman, “Incremental potential contact: intersection- and inversion-free, large-deformation dynamics,” *ACM Transactions on Graphics*, vol. 39, no. 4, 2020.
- [11] T. Möller and B. Trumbore, “Fast, minimum storage ray-triangle intersection,” *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.
- [12] L. S. D. Morley, “The constant-moment plate-bending element,” *Journal of Strain Analysis*, vol. 6, no. 1, pp. 20–24, 1971.
- [13] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [14] A. Paszke *et al.*, “PyTorch: an imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, 2019.
- [15] S. Timoshenko and S. Woinowsky-Krieger, *Theory of Plates and Shells*, 2nd ed. McGraw-Hill, 1959.
- [16] A. van Oosterom and J. Strackee, “The solid angle of a plane triangle,” *IEEE Transactions on Biomedical Engineering*, vol. BME-30, no. 2, pp. 125–126, 1983.
- [17] L. Verlet, “Computer ‘experiments’ on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967.
- [18] P. Virtanen *et al.*, “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.